



National Research University Higher School of Economics

SUSkva

Alexandr Ponkratov, Dmitry Pugachev, Vasilii Silvestrov

2023-11-19

Contest (1)

```
template.cpp
30 lines

#include <bits/stdc++.h>

using namespace std;

#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

using ll = long long;
using ld = long double;
using ull = unsigned long long;

const char en = '\n';

mt19937 rnd(chrono::high_resolution_clock::now().
            time_since_epoch().count());

template <typename T>
int sz(T x) { return (int)x.size(); }

void solve() {}

int32_t main() {
#ifdef LOCAL
    freopen("input.txt", "r", stdin);
#else
    ios_base::sync_with_stdio(0);
    cin.tie(0);
#endif
    solve();
    return 0;
}
```

```
hash.sh
3 lines

# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum |cut -c-6
```

```
CMakeLists.txt
8 lines

set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wshadow -g -
    fsanitize=address,undefined,bounds -D_GLIBCXX_DEBUG")

file(GLOB APP_SOURCES ${CMAKE_CURRENT_SOURCE_DIR}/*.cpp)
foreach(sourcefile ${APP_SOURCES})
    file(RELATIVE_PATH filename ${CMAKE_CURRENT_SOURCE_DIR} ${
        sourcefile})
    string(REPLACE ".cpp" "" file ${filename})
    add_executable(${file} ${sourcefile})
endforeach(sourcefile ${APP_SOURCES})
```

C++ (2)

```
gp-hashtable.cpp
Description: Hash map with mostly the same API as unordered_map, but
~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if
provided).
```

```
e44914, 13 lines

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

const int RANDOM =
```

```
chrono::high_resolution_clock::now().time_since_epoch().
    count();
struct hasher {
    int operator() (int x) const {
        return x ^ RANDOM;
    }
};

gp_hash_table<int, int, hasher> table;
```

```
ordered-set.cpp
Description: A set (not multiset!) with support for finding the n'th ele-
ment, and finding the index of an element. To get a map, change null_type.
Time: O(log(n))
```

```
707789, 26 lines

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, std::less<int>, rb_tree_tag,
            tree_order_statistics_node_update>
            ordered_set;

int main() {
    ordered_set X;
    X.insert(1);
    X.insert(2);
    X.insert(4);
    X.insert(8);
    X.insert(16);

    cout<<*X.find_by_order(1)<<endl; // 2
    cout<<*X.find_by_order(2)<<endl; // 4
    cout<<*X.find_by_order(4)<<endl; // 16
    cout<<(end(X)==X.find_by_order(6))<<endl; // true

    cout<<X.order_of_key(-5)<<endl; // 0
    cout<<X.order_of_key(1)<<endl; // 0
    cout<<X.order_of_key(3)<<endl; // 2
    cout<<X.order_of_key(4)<<endl; // 2
    cout<<X.order_of_key(400)<<endl; // 5
}
```

```
masks.cpp
Description: Iterating through all the submasks and overmasks.
Time: O(3^n)
```

```
b45e1f, 9 lines

for (int submask = mask;; submask = (submask - 1) & mask) {
    // use submask
    if (submask == 0) break;
}

for (int overmask = mask;; overmask = (overmask + 1) | mask) {
    // use overmask
    if (overmask == maxmask) break;
}

struct Query {
    char type;
    int v, u;

    Query(char type) : type(type) {}
```

```
Query(char type, int v, int u) : type(type), v(v), u(u) {}
};

struct DCP {
    int n, k, ans; // n - vertex, k - queries
    vector<int> par, rk;
    vector<pair<pii, int>> hist;
    // 0 - par, 1 - rk, 2 - ans;

    int qL, qR;
    pii edge;
    vector<vector<pii>> tree;
    vector<Query> qs;

    DCP(int _n, int _k) {
        n = ans = _n;
        par.resize(n);
        rk.resize(n, 1);
        for (int i = 0; i < n; ++i) par[i] = i;
        k = _k;
        tree.assign(4 * k, vector<pii>());
    }

    int dsu_get(int v) {
        while (par[v] != v) v = par[v];
        return v;
    }

    void dsu_unite(int a, int b) {
        a = dsu_get(a);
        b = dsu_get(b);
        if (a == b) return;
        if (rk[a] > rk[b]) swap(a, b);
        hist.emplace_back((pii){0, a, par[a]});
        hist.emplace_back((pii){2, -1, ans});
        par[a] = b;
        --ans;
        if (rk[a] == rk[b]) {
            hist.emplace_back((pii){1, b, rk[b]});
            ++rk[b];
        }
    }

    void dsu_unite(pii e) {
        dsu_unite(e.first, e.second);
    }

    void cancel(pair<pii, int> &el) {
        int &type = el.first.first;
        int &id = el.first.second;
        int &val = el.second;
        if (type == 0) {
            par[id] = val;
        } else if (type == 1) {
            rk[id] = val;
        } else if (type == 2) {
            ans = val;
        }
    }

    void add_edge(int _qL, int _qR, pii e) { // [L, R]
        qL = _qL;
        qR = _qR + 1;
        edge = e;
        add_edge_tree(1, 0, k);
    }

    void add_edge_tree(int v, int l, int r) {
        if (qL <= l && r <= qR) {
```

```

        tree[v].emplace_back(edge);
        return;
    }
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    if (qL < m) add_edge_tree(vL, l, m);
    if (m < qR) add_edge_tree(vR, m, r);
}

void go(vector<Query> &_qs) {
    qs = _qs;
    go_tree(1, 0, k);
}

void go_tree(int v, int l, int r) {
    int siz = sz(hist);
    for (auto &e : tree[v]) {
        dsu_unite(e);
    }
    if (l + 1 == r) {
        if (qs[l].type == '?') {
            cout << ans << en;
        }
    } else {
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        go_tree(vL, l, m);
        go_tree(vR, m, r);
    }
    while (sz(hist) > siz) {
        cancel(hist.back());
        hist.pop_back();
    }
}

void solve() {
    int n, k;
    cin >> n >> k;
    DCP dcp(n, k);
    set<pair<pii, int>> edges;
    vector<Query> qs;
    for (int i = 0; i < k; ++i) {
        char tp;
        cin >> tp;
        if (tp == '?') {
            qs.emplace_back(tp);
        } else {
            int v, u;
            cin >> v >> u;
            --v;
            --u;
            if (v > u) swap(v, u);
            qs.emplace_back(tp, v, u);
            if (tp == '+') {
                edges.emplace((pii){v, u}, i);
            } else {
                auto it = edges.lower_bound((pii){v, u}, 0);
                dcp.add_edge(it->second, i, it->first);
                edges.erase(it);
            }
        }
    }
    for (auto &e : edges) {
        dcp.add_edge(e.second, k - 1, e.first);
    }
    if (k) dcp.go(qs);
}

```

linkcut.cpp

Description: Link-cut tree.

ddd3fe, 144 lines

```

struct Node {
    Node *ch[2] = {nullptr, nullptr};
    Node *par = nullptr;
    bool rev = false;
    int val, mn;
    int siz = 1;

    Node() {}

    Node(int val) : val(val), mn(val) {}
};

typedef Node *pnode;

int get_siz(pnode v) {
    return (v == nullptr ? 0 : v->siz);
}

int get_min(pnode v) {
    return (v == nullptr ? INF : v->mn);
}

void update(pnode v) {
    v->siz = 1 + get_siz(v->ch[0]) + get_siz(v->ch[1]);
    v->mn = min(v->val, min(get_min(v->ch[0]), get_min(v->ch[1])));
}

void push(pnode v) {
    if (v == nullptr || !v->rev) {
        return;
    }
    if (v->ch[0] != nullptr) {
        v->ch[0]->rev ^= 1;
    }
    if (v->ch[1] != nullptr) {
        v->ch[1]->rev ^= 1;
    }
    swap(v->ch[0], v->ch[1]);
    v->rev = 0;
}

bool is_root(pnode v) {
    return (v->par == nullptr ||
            (v->par->ch[0] != v && v->par->ch[1] != v));
}

int child_num(pnode v) {
    return (v->par->ch[1] == v);
}

void attach(pnode v, pnode p, int num) {
    if (v != nullptr) {
        v->par = p;
    }
    if (p != nullptr) {
        p->ch[num] = v;
    }
}

void rotate(pnode v) {
    int num = child_num(v);
    pnode p = v->par, vb = v->ch[num ^ 1];
    pnode g = (p == nullptr ? nullptr : p->par);
    if (g != nullptr) {
        if (!is_root(p)) {
            g->ch[child_num(p)] = v;
        }
    }
    if (g->ch[child_num(g)] == p) {
        g->ch[child_num(g)] = v;
    }
    if (p != nullptr) {
        p->par = g;
    }
    if (vb != nullptr) {
        vb->par = v;
    }
    if (v->ch[0] != nullptr) {
        v->ch[0]->par = v;
    }
    if (v->ch[1] != nullptr) {
        v->ch[1]->par = v;
    }
    if (p != nullptr) {
        p->val = v->val;
        p->mn = v->mn;
    }
    v->val = p->val;
    v->mn = p->mn;
    v->siz = p->siz;
    p->siz = 1;
    if (p != nullptr) {
        p->rev = v->rev;
    }
    v->rev = !v->rev;
}

void splay(pnode v) {
    vector<pnode> st;
    pnode cur = v;
    st.emplace_back(cur);
    while (!is_root(cur)) {
        cur = cur->par;
        st.emplace_back(cur);
    }
    for (int i = sz(st) - 1; i >= 0; --i) {
        push(st[i]);
    }
    while (!is_root(v)) {
        if (!is_root(v->par)) {
            if (child_num(v) == child_num(v->par)) {
                rotate(v->par);
            } else {
                rotate(v);
            }
        }
        rotate(v);
    }
}

void expose(pnode v) {
    splay(v);
    v->ch[1] = nullptr;
    update(v);
    while (v->par != nullptr) {
        splay(v->par);
        attach(v, v->par, 1);
        update(v->par);
        splay(v);
    }
}

void make_root(pnode v) {
    expose(v);
    v->rev ^= 1;
}

void link(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    u->par = v;
}

void cut(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    push(u);
    u->ch[1] = nullptr;
    v->par = nullptr;
}

bool is_connected(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    if (is_root(v) && u != v) {
        return false;
    }
}

```

```

        g->ch[child_num(p)] = v;
    }
    if (p != nullptr) {
        p->par = g;
    }
    if (vb != nullptr) {
        vb->par = v;
    }
    if (v->ch[0] != nullptr) {
        v->ch[0]->par = v;
    }
    if (v->ch[1] != nullptr) {
        v->ch[1]->par = v;
    }
    if (p != nullptr) {
        p->val = v->val;
        p->mn = v->mn;
    }
    v->val = p->val;
    v->mn = p->mn;
    v->siz = p->siz;
    p->siz = 1;
    if (p != nullptr) {
        p->rev = v->rev;
    }
    v->rev = !v->rev;
}

void splay(pnode v) {
    vector<pnode> st;
    pnode cur = v;
    st.emplace_back(cur);
    while (!is_root(cur)) {
        cur = cur->par;
        st.emplace_back(cur);
    }
    for (int i = sz(st) - 1; i >= 0; --i) {
        push(st[i]);
    }
    while (!is_root(v)) {
        if (!is_root(v->par)) {
            if (child_num(v) == child_num(v->par)) {
                rotate(v->par);
            } else {
                rotate(v);
            }
        }
        rotate(v);
    }
}

void expose(pnode v) {
    splay(v);
    v->ch[1] = nullptr;
    update(v);
    while (v->par != nullptr) {
        splay(v->par);
        attach(v, v->par, 1);
        update(v->par);
        splay(v);
    }
}

void make_root(pnode v) {
    expose(v);
    v->rev ^= 1;
}

void link(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    u->par = v;
}

void cut(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    push(u);
    u->ch[1] = nullptr;
    v->par = nullptr;
}

bool is_connected(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    if (is_root(v) && u != v) {
        return false;
    }
}

```

```

    } else {
        return true;
    }
}

```

```

int get_min(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    return get_min(u);
}

```

segtreebeats.cpp

Description: Segment Tree beats (TODO: WRITE OPERATIONS).

92635a, 228 lines

```

struct SegTree {
    struct Node {
        ll max, sec_max;
        int cnt_max;
        ll min, sec_min;
        int cnt_min;
        ll sum;
        ll push_add;
        ll push_eq;
    };

    int n, qL, qR;
    ll val, ans;

    vector<Node> tree;
    vector<ll> a;

    SegTree(int _n) {
        n = _n;
        tree.assign(4 * n, {0, -INFL, 1, 0, INFL, 1, 0, 0, -1});
    }

    void update_vertex(int v, int l, int r) {
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;

        tree[v].sum = tree[vL].sum + tree[vR].sum;

        tree[v].max = max(tree[vL].max, tree[vR].max);
        tree[v].sec_max = max(tree[vL].sec_max, tree[vR].sec_max);
        tree[v].cnt_max = 0;
        if (tree[vL].max == tree[v].max) {
            tree[v].cnt_max += tree[vL].cnt_max;
        } else {
            tree[v].sec_max = max(tree[v].sec_max, tree[vL].max);
        }
        if (tree[vR].max == tree[v].max) {
            tree[v].cnt_max += tree[vR].cnt_max;
        } else {
            tree[v].sec_max = max(tree[v].sec_max, tree[vR].max);
        }

        tree[v].min = min(tree[vL].min, tree[vR].min);
        tree[v].sec_min = min(tree[vL].sec_min, tree[vR].sec_min);
        tree[v].cnt_min = 0;
        if (tree[vL].min == tree[v].min) {
            tree[v].cnt_min += tree[vL].cnt_min;
        } else {
            tree[v].sec_min = min(tree[v].sec_min, tree[vL].min);
        }
    }
}

```

```

    if (tree[vR].min == tree[v].min) {
        tree[v].cnt_min += tree[vR].cnt_min;
    } else {
        tree[v].sec_min = min(tree[v].sec_min, tree[vR].min);
    }
}

void recalc_eq(int v, int l, int r, ll cur) {
    tree[v].max = tree[v].min = tree[v].push_eq = cur;
    tree[v].sec_max = -INFL;
    tree[v].sec_min = INFL;
    tree[v].cnt_max = tree[v].cnt_min = r - l;
    tree[v].sum = cur * (ll) (r - l);
    tree[v].push_add = 0;
}

void recalc_add(int v, int l, int r, ll cur) {
    if (tree[v].min == tree[v].max) {
        recalc_eq(v, l, r, tree[v].max + cur);
        return;
    }
    tree[v].max += cur;
    if (tree[v].sec_max != -INFL) {
        tree[v].sec_max += cur;
    }

    tree[v].min += cur;
    if (tree[v].sec_min != INFL) {
        tree[v].sec_min += cur;
    }

    tree[v].sum += (ll) (r - l) * cur;
    tree[v].push_add += cur;
}

void recalc_min(int v, int l, int r, ll cur) {
    if (tree[v].min >= cur) {
        recalc_eq(v, l, r, cur);
        return;
    }
    if (tree[v].max <= cur) return;
    if (tree[v].sec_min == tree[v].max) {
        tree[v].sec_min = cur;
    }
    tree[v].sum -= (ll) (tree[v].max - cur) *
        (ll) (tree[v].cnt_max);
    tree[v].max = cur;
}

void recalc_max(int v, int l, int r, ll cur) {
    if (tree[v].max <= cur) {
        recalc_eq(v, l, r, cur);
        return;
    }
    if (tree[v].min >= cur) return;
    if (tree[v].sec_max == tree[v].min) {
        tree[v].sec_max = cur;
    }
    tree[v].sum += (ll) (tree[v].max - cur) *
        (ll) (tree[v].cnt_max);
    tree[v].min = cur;
}

void push(int v, int l, int r) {
    if (l + 1 == r) return;
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    if (tree[v].push_eq != -1) {
        recalc_eq(vL, l, m, tree[v].push_eq);
    }
}

```

```

        recalc_eq(vR, m, r, tree[v].push_eq);
        tree[v].push_eq = -1;
        return;
    }

    recalc_add(vL, l, m, tree[v].push_add);
    recalc_add(vR, m, r, tree[v].push_add);
    tree[v].push_add = 0;

    recalc_min(vL, l, m, tree[v].max);
    recalc_min(vR, m, r, tree[v].max);

    recalc_max(vL, l, m, tree[v].min);
    recalc_max(vR, m, r, tree[v].min);
}

void build(vector<ll> &a) {
    a = _a;
    build_tree(1, 0, n);
}

void build_tree(int v, int l, int r) {
    if (l + 1 == r) {
        tree[v] = {a[l], -INFL, 1, a[l], INFL, 1, a[l], 0, -1};
        return;
    }
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    build_tree(vL, l, m);
    build_tree(vR, m, r);
    update_vertex(v, l, r);
}

void update_segment_min(int _qL, int _qR, ll _val) {
    qL = _qL;
    qR = _qR + 1;
    val = _val;
    update_segment_min_tree(1, 0, n);
}

void update_segment_min_tree(int v, int l, int r) {
    if (tree[v].max <= val) return;
    if (qL <= l && r <= qR && tree[v].sec_max < val) {
        recalc_min(v, l, r, val);
        return;
    }
    push(v, l, r);
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    if (qL < m) update_segment_min_tree(vL, l, m);
    if (m < qR) update_segment_min_tree(vR, m, r);
    update_vertex(v, l, r);
}

void update_segment_max(int _qL, int _qR, ll _val) {
    qL = _qL;
    qR = _qR + 1;
    val = _val;
    update_segment_max_tree(1, 0, n);
}

void update_segment_max_tree(int v, int l, int r) {
    if (tree[v].min >= val) return;
    if (qL <= l && r <= qR && tree[v].sec_min > val) {
        recalc_max(v, l, r, val);
        return;
    }
    push(v, l, r);
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    if (qL < m) update_segment_max_tree(vL, l, m);
}

```

```
        if (m < qR) update_segment_max_tree(vR, m, r);
        update_vertex(v, l, r);
    }

    void update_segment_add(int _qL, int _qR, ll _val) {
        qL = _qL;
        qR = _qR + 1;
        val = _val;
        update_segment_add_tree(l, 0, n);
    }

    void update_segment_add_tree(int v, int l, int r) {
        if (qL <= l && r <= qR) {
            recalc_add(v, l, r, val);
            return;
        }
        push(v, l, r);
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        if (qL < m) update_segment_add_tree(vL, l, m);
        if (m < qR) update_segment_add_tree(vR, m, r);
        update_vertex(v, l, r);
    }

    ll get(int _qL, int _qR) {
        qL = _qL;
        qR = _qR + 1;
        ans = 0;
        get_tree(l, 0, n);
        return ans;
    }

    void get_tree(int v, int l, int r) {
        if (qL <= l && r <= qR) {
            ans += tree[v].sum;
            return;
        }
        push(v, l, r);
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        if (qL < m) get_tree(vL, l, m);
        if (m < qR) get_tree(vR, m, r);
    }
};
```

cht.cpp

Description: Convex-hull trick.

7e474f, 49 lines

```
struct Line {
    ll k = 0, b = -INFL;
    Line() = default;
    Line(ll k, ll b) : k(k), b(b){};

    ld operator()(ld x) {
        return k * x + b;
    }
};

bool operator<(Line a, Line b) {
    return a.k < b.k || (a.k == b.k && a.b > b.b);
}

bool operator<(pair<Line, ld> a, pair<Line, ld> b) {
    return a.second < b.second;
}

struct CHT {
    vector<pair<Line, ld>> convex;

    void add(Line a) {
```

```
        while (!convex.empty() && a(convex.back().second) >
            convex.back().first(convex.back().second)) {
            convex.pop_back();
        }
        ld xn = -INFL;
        if (!convex.empty()) {
            xn = (a.b - convex.back().first.b + 0.0) / (convex.
                back().first.k - a.k);
        }
        convex.push_back({a, xn});
    }

    CHT(vector<Line> lines) {
        sort(all(lines));
        for (int i = 0; i < (int) lines.size(); i) {
            int j = i;
            while (j < (int) lines.size() && lines[i].k ==
                lines[j].k) {
                ++j;
            }
            add(lines[i]);
            i = j;
        }
    }

    ld get(ld x) {
        auto it = upper_bound(all(convex), pair<Line(), x) -
            convex.begin();
        return convex[it - 1].first(x);
    }
};
```

lichao.cpp

Description: Li-Chao tree.

107896, 70 lines

```
struct LiChao {
    struct Line {
        ll k = 0, b = -INFL;

        Line() = default;

        Line(ll k, ll b) : k(k), b(b){};

        ll operator()(ll x) {
            return k * x + b;
        }
    };

    struct Node {
        Node *l = nullptr, *r = nullptr;
        Line cur;

        Node() = default;
    };

    Node *root = nullptr;
    int n = 1e9 + 1;

    void make_node(Node *&v) {
        if (v == nullptr) {
            v = new Node();
        }
    }

    void add(Node *&v, int l, int r, Line cur) {
        make_node(v);
        int m = (l + r) / 2;
        if (cur(m) > v->cur(m)) {
            swap(cur, v->cur);
        }
    }
```

```
    }

    if (l + 1 == r) {
        return;
    }

    if (cur(l) > v->cur(l)) {
        add(v->l, l, m, cur);
    } else {
        add(v->r, m, r, cur);
    }
}

void add(Line cur) {
    add(root, 0, n, cur);
}

ll get(Node *v, int l, int r, int x) {
    if (v == nullptr) {
        return -INFL;
    }
    ll ans = v->cur(x);
    if (l + 1 == r) {
        return ans;
    }

    int m = (l + r) / 2;
    if (x < m) {
        ans = max(ans, get(v->l, l, m, x));
    } else {
        ans = max(ans, get(v->r, m, r, x));
    }

    return ans;
}

ll get(int x) {
    return get(root, 0, n, x);
}
};
```

Graph (4)

dominator-tree.cpp

Description: Dominator tree.

154ba6, 165 lines

```
struct Edge {
    int from, to, id;

    Edge() = default;
    Edge(int from, int to, int id) : from(from), to(to), id(id)
        {}
};

struct DSU {
    int n;
    vector<int> par;
    vector<pii> mn;

    DSU() = default;
    DSU(int n) : n(n) {
        par.resize(n);
        mn.resize(n);
        init();
    }

    void init() {
        for (int i = 0; i < n; ++i) {
            par[i] = i;
            mn[i] = {INF, i};
        }
    }
```

```

int get(int v) {
    if (par[v] == v) {
        return v;
    }
    int p = get(par[v]);
    mn[v] = min(mn[v], mn[par[v]]);
    if (mn[par[v]].first < mn[v].first) {
        mn[v] = mn[par[v]];
    }
    par[v] = p;
    return p;
}

//

void unite(int a, int b) {
    par[a] = b;
}

};

struct DominatorTree {
    int n;
    vector<Edge> edges;
    vector<int> sdom, idom, tin, order, par, used, dp;
    vector<vector<int>> g, rg, queries;
    DSU dsu_sdom, dsu_idom;

    DominatorTree() = default;
    DominatorTree(int n) : n(n), dsu_sdom(n), dsu_idom(n) {
        sdom.resize(n, INF); // semi-dominator
        idom.resize(n, INF); // immediate dominator
        tin.resize(n, -1);
        par.resize(n);
        used.resize(n, 0);
        dp.resize(n, INF);
        g.resize(n);
        rg.resize(n);
        queries.resize(n);
    }

    void add_edge(int from, int to) {
        edges.emplace_back(from, to, sz(edges));
    }

    void dfs(int v) {
        tin[v] = sz(order);
        order.emplace_back(v);
        for (auto i: g[v]) {
            const auto &e = edges[i];
            if (tin[e.to] == -1) {
                par[e.to] = v;
                dfs(e.to);
            }
        }
    }

    void dfs_idom(int v) {
        used[v] = 1;
        for (auto i: g[v]) {
            const auto &e = edges[i];
            if (!used[e.to]) {
                dfs_idom(e.to);
            }
        }
        for (auto u: queries[v]) {
            dsu_idom.get(u);
            dp[u] = dsu_idom.mn[u].second;
        }
        dsu_idom.mn[v] = {sdom[v], v};
        for (auto i: g[v]) {

```

```

            const auto &e = edges[i];
            if (par[e.to] == v) {
                dsu_idom.unite(e.to, v);
            }
        }
    }

    void build(int s) {
        for (int i = 0; i < sz(edges); ++i) {
            g[edges[i].from].emplace_back(i);
            rg[edges[i].to].emplace_back(i);
        }
        // reorder vertex
        dfs(s);

        // build sdom
        for (int _ = sz(order) - 1; _ >= 0; --_) {
            int v = order[_];
            if (v == s) {
                continue;
            }
            for (auto i: rg[v]) {
                const auto &e = edges[i];
                if (tin[e.from] == -1) {
                    continue;
                }
                if (tin[e.from] < tin[v]) {
                    sdom[v] = min(sdom[v], tin[e.from]);
                }
                else {
                    int u = dsu_sdom.get(e.from);
                    sdom[v] = min(sdom[v], dsu_sdom.mn[e.from].first);
                }
            }
            dsu_sdom.mn[v] = {sdom[v], v};
            for (auto i: g[v]) {
                const auto &e = edges[i];
                if (v == par[e.to]) {
                    dsu_sdom.unite(e.to, v);
                }
            }
        }

        // build queries for idoms
        for (int i = 0; i < n; ++i) {
            if (i == s || sdom[i] == INF || tin[i] == -1) {
                continue;
            }
            queries[order[sdom[i]]].emplace_back(i);
        }
        dfs_idom(s);

        // build idom
        idom[s] = tin[s];
        for (auto v: order) {
            if (v == s) {
                continue;
            }
            if (v == dp[v]) {
                idom[v] = sdom[v];
            }
            else {
                idom[v] = idom[dp[v]];
            }
        }
    }

    int get_idom(int v) {

```

```

        return (idom[v] == INF ? -1 : order[idom[v]]);
    }
};

```

eulerian-path.cpp

Description: Finds eulerian path.

687663, 16 lines

```

vector<vector<int>> g;
vector<bool> used;
vector<int> ed, tour;

void dfs(int v) {
    while (!g[v].empty()) {
        int u = g[v].back();
        g[v].pop_back();
        if (used[u]) {
            continue;
        }
        used[u] = 1;
        dfs(ed[u] ^ v);
    }
    tour.push_back(v);
}

```

hld.cpp

Description: Heavy-light decomposition.

Time: $\mathcal{O}(\log^2(n))$ per query

98d176, 92 lines

```

struct HLD {

    struct SegTree {};

    int n, T;
    SegTree st;
    vector<vector<int>> tree;
    vector<int> par, siz, tin, tout, head;
    ll ans;

    HLD(int _n) {
        n = _n;
        tree.resize(n);
        par.resize(n, -1);
        siz.resize(n, 0);
        tin.resize(n);
        tout.resize(n);
        head.resize(n);
        st = SegTree(n);
    }

    void add_edge(int v, int u) {
        tree[v].emplace_back(u);
        tree[u].emplace_back(v);
    }

    void build(int v = 0) {
        dfs_siz(v, v);
        T = 0;
        head[v] = v;
        dfs_hld(v, v);
    }

    void dfs_siz(int v, int p) {
        par[v] = p;
        siz[v] = 1;
        for (auto &u : tree[v]) {
            if (u != p) {
                dfs_siz(u, v);
                siz[v] += siz[u];
            }
        }
    }

```

```

    }
    for (int i = 0; i < sz(tree[v]); ++i) {
        int x = tree[v][0], u = tree[v][i];
        if (x == p || siz[u] > siz[x]) {
            swap(tree[v][0], tree[v][i]);
        }
    }
}

void dfs_hld(int v, int p) {
    tin[v] = T++;
    for (auto u : tree[v]) {
        if (u == p) {
            continue;
        }
        if (u == tree[v][0]) {
            head[u] = head[v];
        } else {
            head[u] = u;
        }
        dfs_hld(u, v);
    }
    tout[v] = T;
}

void update(int v, int val) {
    st.update_segment(tin[v], tin[v], val);
}

bool is_anc(int v, int u) {
    return tin[v] <= tin[u] && tout[u] <= tout[v];
}

void go_up(int &v, int u) {
    while (!is_anc(head[v], u)) {
        ans = max(ans, st.get(tin[head[v]], tin[v]));
        v = par[head[v]];
    }
}

ll get(int v, int u) { // max on path
    ans = -INFL;
    go_up(v, u);
    go_up(u, v);
    if (!is_anc(v, u)) {
        swap(v, u);
    }
    ans = max(ans, st.get(tin[v], tin[u]));
    return ans;
}

};

```

kuhn.cpp

Description: Kuhn's algorithm.

b95c30, 27 lines

```

vector<vector<int>> g;
vector<int> used, mt;
int timer = 1;
bool dfs(int v) {
    if (used[v] == timer) {
        return false;
    }
    used[v] = timer;
    for (auto u : g[v]) {
        if (mt[u] == -1) {
            mt[u] = v;
            return true;
        }
    }
}

```

```

    for (auto u : g[v]) {
        if (dfs(mt[u])) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

for (int i = 0; i < n; ++i) {
    if (dfs(i)) {
        ++timer;
    }
}

```

maxflow.cpp

Description: Dinic's algorithm

Time: $O(V^2E)$

608498, 118 lines

```

struct MaxFlow {
    struct Edge {
        ll flow, cap;
        int to, id;

        Edge() {}

        Edge(ll flow, ll cap, int to, int id) : flow(flow), cap
            (cap), to(to), id(id) {}
    };

    int n;
    vector<vector<Edge>> g;
    vector<int> d, head, used;
    ll max_cap;
    int s, t;

    MaxFlow() {}

    MaxFlow(int _n) {
        n = _n;
        g.resize(n);
    }

    void add_edge(int from, int to, ll cap) {
        g[from].emplace_back(0, cap, to, sz(g[to]));
        g[to].emplace_back(0, 0, from, sz(g[from]) - 1);
    }

    bool bfs() {
        d.assign(n, INF);
        d[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (auto e : g[v]) {
                if (d[e.to] == INF && e.cap - e.flow >= max_cap) {
                    d[e.to] = d[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return d[t] != INF;
    }

    ll dfs(int v, ll cur_flow) {
        if (v == t) {
            return cur_flow;

```

```

        }
        for (; head[v] < sz(g[v]); ++head[v]) {
            auto &e = g[v][head[v]];
            if (e.cap - e.flow >= max_cap && d[v] + 1 == d[e.to]) {
                ll new_flow = dfs(e.to, min(cur_flow, e.cap - e
                    .flow));
                if (new_flow) {
                    e.flow += new_flow;
                    g[e.to][e.id].flow -= new_flow;
                    return new_flow;
                }
            }
        }
        return 0;
    }

    ll find_max_flow(int _s, int _t) {
        s = _s;
        t = _t;
        ll res = 0;
        for (int k = 30; k >= 0; --k) {
            max_cap = (1 << k);
            while (bfs()) {
                head.assign(n, 0);
                ll flow = 0;
                do {
                    flow = dfs(s, INFL);
                    res += flow;
                } while (flow);
            }
        }
        return res;
    }

    ll dfs_const_flow(int v, ll cur_flow) {
        used[v] = 1;
        if (v == t) {
            return cur_flow;
        }
        for (auto &e : g[v]) {
            if (!used[e.to] && e.cap - e.flow > 0) {
                ll new_flow = dfs_const_flow(e.to, min(cur_flow
                    , e.cap - e.flow));
                if (new_flow) {
                    e.flow += new_flow;
                    g[e.to][e.id].flow -= new_flow;
                    return new_flow;
                }
            }
        }
        return 0;
    }

    bool find_const_flow(int _s, int _t, ll F) {
        s = _s;
        t = _t;
        ll res = 0, flow = 0;
        max_cap = F;
        do {
            used.assign(n, 0);
            flow = dfs_const_flow(s, INF);
            res += flow;
        } while (flow && res < F);
        return res == F;
    }

    ll get_edge_flow(int v, int id) {
        return g[v][id].flow;
    }
}

```

```
    }
};

mincost.cpp
Description: Similar to the Edmonds-Karp.
Time:  $\mathcal{O}(VE + FE \log(V))$ 
126796, 90 lines

struct MinCostMaxFlow {
    struct Edge {
        ll flow, cap, price;
        int to, id;

        Edge() {}

        Edge(ll flow, ll cap, ll price, int to, int id) : flow(
            flow), cap(cap), price(price), to(to), id(id) {}
    };

    int n;
    int s, t;
    ll ans;
    vector<vector<Edge>> g;
    vector<int> d;
    vector<ll> add_f;
    vector<pii> par;

    MinCostMaxFlow() {}

    MinCostMaxFlow(int _n) {
        n = _n;
        g.resize(n);
    }

    void add_edge(int from, int to, ll cap, ll price) {
        g[from].emplace_back(0, cap, price, to, sz(g[to]));
        g[to].emplace_back(0, 0, -price, from, sz(g[from]) - 1)
        ;
    }

    ll get_edge_flow(int v, int id) {
        return g[v][id].flow;
    }

    void FB() {
        d.assign(n, INF);
        add_f.assign(n, 0);
        par.assign(n, {-1, -1});
        d[s] = 0;
        add_f[0] = INF;
        queue<int> q;
        q.push(s);
        vector<int> used(n, 0);
        used[s] = 1;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            used[v] = 0;
            for (int i = 0; i < sz(g[v]); ++i) {
                auto &e = g[v][i];
                if (e.flow < e.cap && d[e.to] > d[v] + e.price)
                {
                    d[e.to] = d[v] + e.price;
                    add_f[e.to] = min(add_f[v], e.cap - e.flow)
                    ;
                    par[e.to] = {v, i};
                    if (!used[e.to]) {
                        q.push(e.to);
                        used[e.to] = 1;
                    }
                }
            }
        }
    }
};
```

```
    }
    }
}

void push_flow(ll flow) {
    int cur = t;
    while (cur != s) {
        int prev = par[cur].first, id = par[cur].second;
        g[prev][id].flow += flow;
        g[cur][g[prev][id].id].flow -= flow;
        ans += g[prev][id].price * flow;
        cur = prev;
    }
}

ll min_cost_max_flow(int _s, int _t) {
    ans = 0;
    s = _s;
    t = _t;
    while (true) {
        FB();
        ll flow = add_f[t];
        if (flow == 0) {
            break;
        }
        push_flow(flow);
    }
    return ans;
}

};
```

Strings (5)

```
z-func.cpp
Description: Z-function.
11739d, 18 lines

vector<int> z_function(string &s) {
    int n = (int) s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) {
            z[i] = min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            r = i + z[i] - 1;
            l = i;
        }
    }
    return z;
}

prefix-func.cpp
Description: Prefix function.
8cb94c, 14 lines

vector<int> prefix_function(string &s) {
    int n = (int) s.size();
    vector<int> p(n, 0);
    for (int i = 1; i < n; i++) {
        int cur = p[i - 1];
        while (s[i] != s[cur] && cur > 0) {
            cur = p[cur - 1];
        }
        if (s[i] == s[cur]) {

```

```
            p[i] = cur + 1;
        }
    }
    return p;
}

sufarray.cpp
Description: Suffix array and LCP.
Time:  $\mathcal{O}(n \log(n))$  and  $\mathcal{O}(n)$ 
efc54c, 71 lines

int mod(int x, int m) {
    if (x < 0) x += m;
    if (x >= m) x -= m;
    return x;
}

vector<int> sufix_array(string s) {
    s += '$';
    int n = sz(s);
    vector<int> arr(n), narr(n), head(n), c(n), nc(n);
    for (int i = 0; i < n; ++i) {
        arr[i] = i;
    }
    sort(all(arr), [&](int x, int y) {
        return s[x] < s[y];
    });
    int cl = 0;
    c[arr[0]] = cl;
    head[0] = 0;
    for (int i = 1; i < n; ++i) {
        if (s[arr[i]] != s[arr[i - 1]]) {
            head[++cl] = i;
        }
        c[arr[i]] = cl;
    }
    for (int k = 1; k < n && cl < n; k *= 2) {
        for (int i = 0; i < n; i++) {
            int j = mod(arr[i] - k, n);
            narr[head[c[j]]++] = j;
        }
        cl = 0;
        head[0] = 0;
        pii prev = {c[narr[0]], c[mod(narr[0] + k, n)]};
        for (int i = 1; i < n; i++) {
            pii cur = {c[narr[i]], c[mod(narr[i] + k, n)]};
            if (cur != prev) {
                head[++cl] = i;
            }
            nc[narr[i]] = cl;
            prev = cur;
        }
        swap(arr, narr);
        swap(c, nc);
    }
    // returns sufux array without $
    return vector<int>(arr.begin() + 1, arr.end());
}

vector<int> build_lcp(string &s, vector<int> &suf) {
    int n = sz(s);
    vector<int> lcp(n - 1, order(n));
    for (int i = 0; i < n; ++i) {
        order[suf[i]] = i;
    }
    int l = 0;
    for (int i = 0; i < n; ++i) {
        int id = order[i];
        if (id + 1 == n) {
            l = 0;

```



```
        continue;
    }
    int j = suf[id + 1];
    if (l) --l;
    while (max(i + 1, j + 1) < n &&
           s[i + 1] == s[j + 1]) {
        ++l;
    }
    lcp[id] = l;
}
return lcp;
}
```

sufavtomat.cpp
Description: Suffix automaton. 23f122, 51 lines

```
struct Node {
    int go[26];
    int suf, prev, term, len;

    Node() {
        for (auto &i : go) {
            i = -1;
        }
        len = 0;
        suf = -1;
        prev = -1;
        term = 0;
    }
};

vector<Node> automat;

int add(int a, int ch) {
    int b = sz(automat);
    automat.emplace_back();
    automat[b].prev = a;
    automat[b].suf = 0;
    automat[b].len = automat[a].len + 1;
    for (; a != -1; a = automat[a].suf) {
        if (automat[a].go[ch] == -1) {
            automat[a].go[ch] = b;
            continue;
        }
        int c = automat[a].go[ch];
        if (automat[c].prev == a) {
            automat[b].suf = c;
            break;
        }
    }
    int d = sz(automat);
    automat.emplace_back();
    automat[d].suf = automat[c].suf;
    automat[d].len = automat[a].len + 1;
    automat[c].suf = d;
    automat[b].suf = d;
    automat[d].prev = a;
    for (int i = 0; i < 26; ++i) {
        automat[d].go[i] = automat[c].go[i];
    }
    for (; a != -1 && automat[a].go[ch] == c; a = automat[a].suf) {
        automat[a].go[ch] = d;
    }
    break;
}
// returns id of the added vertex
return b;
}
```

korasik.cpp
Description: Aho–Corasick algorithm. 9682e1, 62 lines

```
struct Node {
    int par;
    int par_c;
    int go[26];
    int term;
    int link;
    int super;
    int cnt;
};

const int N = 1e6 + 5;
int ptr = 1;
Node trie[N];

void add(string s) {
    int cur = 1;
    for (auto &q : s) {
        int c = q - 'a';
        if (trie[cur].go[c]) {
            cur = trie[cur].go[c];
        } else {
            ++ptr;
            trie[cur].go[c] = ptr;
            trie[ptr].par = cur;
            trie[ptr].par_c = c;
            cur = ptr;
        }
    }
    trie[cur].cnt++;
}

void build() {
    queue<int> q;
    q.push(1);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        if (v != 1) {
            if (trie[v].par == 1) {
                trie[v].link = trie[v].super = 1;
            } else {
                trie[v].link = trie[trie[trie[v].par].link].go[
                    trie[v].par_c];
                trie[v].super = (trie[trie[v].link].cnt ? trie[
                    v].link : trie[trie[v].link].super);
            }
        }
        trie[v].cnt += trie[trie[v].link].cnt;
        for (int c = 0; c < 26; c++) {
            if (trie[v].go[c]) {
                q.push(trie[v].go[c]);
            } else {
                if (v == 1) {
                    trie[v].go[c] = 1;
                } else {
                    trie[v].go[c] = trie[trie[v].link].go[c];
                }
            }
        }
    }
}
```

```
trie[1].link = trie[1].super = trie[1].par = 1;
build();
```

manacher.cpp
Description: Manacher's algorithm. ab7999, 28 lines

```
vector<int> manacher_odd(string s) {
    int n = (int) s.size();
    vector<int> d(n, 1);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r)
            d[i] = min(r - i + 1, d[l + r - i]);
        while (i - d[i] >= 0 && i + d[i] < n && s[i - d[i]] ==
               s[i + d[i]])
            d[i]++;
        if (i + d[i] - 1 > r)
            l = i - d[i] + 1, r = i + d[i] - 1;
    }
    return d;
}

vector<int> manacher_even(string s) {
    int n = (int) s.size();
    vector<int> d(n, 0);
    int l = -1, r = -1;
    for (int i = 0; i < n - 1; i++) {
        if (i < r)
            d[i] = min(r - i, d[l + r - i - 1]);
        while (i - d[i] >= 0 && i + d[i] + 1 < n && s[i - d[i]]
               == s[i + d[i] + 1])
            d[i]++;
        if (i + d[i] > r)
            l = i - d[i] + 1, r = i + d[i];
    }
    return d;
}
```

Math (6)

fft.cpp
Description: FFT. 5f442d, 126 lines

```
constexpr int P = 998244353;
using i64 = long long;

// assume -P <= x < 2P
int norm(int x) {
    if (x < 0) {
        x += P;
    }
    if (x >= P) {
        x -= P;
    }
    return x;
}

template<class T>
T power(T a, int b) {
    T res = 1;
    for (; b; b /= 2, a *= a) {
        if (b % 2) {
            res *= a;
        }
    }
    return res;
}

struct Z {
    int x;
    Z(int x = 0) : x(norm(x)) {}
    int val() const {
        return x;
    }
};
```

```

}
Z operator-() const {
    return Z(norm(P - x));
}
Z inv() const {
    assert(x != 0);
    return power(*this, P - 2);
}
Z &operator*=(const Z &rhs) {
    x = i64(x) * rhs.x % P;
    return *this;
}
Z &operator+=(const Z &rhs) {
    x = norm(x + rhs.x);
    return *this;
}
Z &operator-=(const Z &rhs) {
    x = norm(x - rhs.x);
    return *this;
}
Z &operator/=(const Z &rhs) {
    return *this *= rhs.inv();
}
friend Z operator*(const Z &lhs, const Z &rhs) {
    Z res = lhs;
    res *= rhs;
    return res;
}
friend Z operator+(const Z &lhs, const Z &rhs) {
    Z res = lhs;
    res += rhs;
    return res;
}
friend Z operator-(const Z &lhs, const Z &rhs) {
    Z res = lhs;
    res -= rhs;
    return res;
}
friend Z operator/(const Z &lhs, const Z &rhs) {
    Z res = lhs;
    res /= rhs;
    return res;
}
}

std::vector<int> rev;
std::vector<Z> roots{0, 1};
void dft(std::vector<Z> &a) {
    int n = a.size();

    if (int(rev.size()) != n) {
        int k = __builtin_ctz(n) - 1;
        rev.resize(n);
        for (int i = 0; i < n; i++) {
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
        }
    }

    for (int i = 0; i < n; i++) {
        if (rev[i] < i) {
            std::swap(a[i], a[rev[i]]);
        }
    }

    if (int(roots.size()) < n) {
        int k = __builtin_ctz(roots.size());
        roots.resize(n);
        while ((1 << k) < n) {
            Z e = power(Z(3), (P - 1) >> (k + 1));
            for (int i = 1 << (k - 1); i < (1 << k); i++) {

```

```

        roots[2 * i] = roots[i];
        roots[2 * i + 1] = roots[i] * e;
    }
    k++;
}
}
for (int k = 1; k < n; k *= 2) {
    for (int i = 0; i < n; i += 2 * k) {
        for (int j = 0; j < k; j++) {
            Z u = a[i + j];
            Z v = a[i + j + k] * roots[k + j];
            a[i + j] = u + v;
            a[i + j + k] = u - v;
        }
    }
}
}

void idft(std::vector<Z> &a) {
    int n = a.size();
    std::reverse(a.begin() + 1, a.end());
    dft(a);
    Z inv = (1 - P) / n;
    for (int i = 0; i < n; i++) {
        a[i] *= inv;
    }
}
}

```

factorization.cpp

Description: Pollard's rho algorithm.

Time: $\mathcal{O}(\sqrt[3]{n})$

68092c, 107 lines

```

ll binpow(ll a, ll n, ll mod) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        int128_t b = binpow(a, n / 2, mod);
        return (b * b) % mod;
    }
    return (((int128_t) a) * binpow(a, n - 1, mod)) % mod;
}

```

```

constexpr int N = 1e7;
vector<int> pr, lp;

```

```

bool prime(ll n) {
    if (n <= N) {
        return binary_search(all(pr), n);
    }
    int iter = 60;
    int s = 0;
    ll d = n - 1;
    while (d % 2 == 0) {
        d /= 2;
        ++s;
    }
    auto test = [&](ll a) {
        if (binpow(a, d, n) == 1) {
            return true;
        }
    }
    ll _2r = 1;
    for (int r = 0; r < s; ++r) {
        auto tmp = binpow(binpow(a, d, n), _2r, n);
        if (tmp == n - 1) {
            return true;
        }
        _2r *= 2;
    }
}

```

```

    return false;
};
for (int _ = 0; _ < iter; ++_) {
    ll a = uniform_int_distribution<ll>(1, n - 1)(rnd);
    if (!test(a)) {
        return false;
    }
}
return true;
}

ll f(ll x, ll n) {
    return ((int128_t) x * (int128_t) x + (int128_t) 2) % n;
}

ll pollard(ll n) {
    ll a = uniform_int_distribution<ll>(0, n - 1)(rnd);
    ll x = a, y = a, d = 1;
    constexpr int iter = 5e4;
    for (int _ = 0; _ < iter; ++_) {
        x = f(f(x, n), n);
        y = f(y, n);
        d = gcd(abs(x - y), n);
        if (d != 1 && d != n) {
            break;
        }
    }
    if (d == 1 || d == n) {
        pollard(n);
    }
    return d;
}

vector<ll> res;

void factor(ll n) {
    if (n <= N) {
        while (true) {
            if (lp[n] == 0) {
                break;
            }
            res.push_back(lp[n]);
            n /= lp[n];
        }
        return;
    }
    if (prime(n)) {
        res.push_back(n);
        return;
    }
    ll d = pollard(n);
    factor(n / d);
    factor(d);
}

void solve() {
    ll n;
    cin >> n;
    lp.resize(N + 1);
    for (int i = 2; i <= N; ++i) {
        if (lp[i] == 0) {
            lp[i] = i;
            pr.push_back(i);
        }
        for (int j = 0; j < (int) pr.size() && pr[j] <= lp[i]
            && i * pr[j] <= N; ++j) {
            lp[i * pr[j]] = pr[j];
        }
    }
}

```

```
    factor(n);
}

crt.cpp
Description: Chinese Remainder Theorem.
812c50, 72 lines

struct Eq {
    // x = a (mod m)
    ll a, m;

    Eq(){};

    Eq(ll a, ll m) : a(a), m(m){};
};

ll binpow(ll a, ll n, ll m) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        int128_t b = binpow(a, n / 2, m);
        return (b * b) % m;
    }
    int128_t x = binpow(a, n - 1, m);
    return (a * x) % m;
}

ll binpow(ll a, ll n) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        ll b = binpow(a, n / 2);
        return b * b;
    }
    return a * binpow(a, n - 1);
}

ll phi(ll a) {
    ll d = 2, k = a;
    map<ll, int> cnt;
    while (d * d <= a) {
        if (k % d == 0) {
            k /= d;
            ++cnt[d];
        } else {
            ++d;
        }
    }
    if (k != 1) {
        ++cnt[k];
    }
    ll ans = 1;
    for (auto i : cnt) {
        ans *= binpow(i.first, i.second - 1) * (i.first - 1);
    }
    return ans;
}

ll gcd(ll a, ll b) {
    return std::gcd(abs(a), abs(b));
}

Eq solve(Eq ai, Eq bi) {
    if (ai.m == -1 || bi.m == -1) {
        return {0, -1};
    }
    ll a = ai.m, b = bi.m, c = ai.a - bi.a;
    ll d = ::gcd(a, b);
    a /= d;
```

```
    b /= d;
    if (c % d != 0) {
        return {0, -1};
    }
    c /= d;
    ll x = (((int128_t) -c * (int128_t) binpow(a, phi(b) - 1, b
        )) % b + b) % b;
    x = ((int128_t) ai.m * (int128_t) x + ai.a) % lcm(ai.m, bi.
        m);
    return {x, lcm(ai.m, bi.m)};
}

ext-gcd.cpp
Description: Extended GCD.
6e27a2, 71 lines

template<typename T>
T extgcd(T a, T b, T &x, T &y) {
    if (a == 0) {
        x = 0;
        y = 1;
        return b;
    }
    T p = b / a;
    T g = extgcd(b - p * a, a, y, x);
    x -= p * y;
    return g;
}

template<typename T>
bool diophantine(T a, T b, T c, T &x, T &y, T &g) {
    if (a == 0 && b == 0) {
        if (c == 0) {
            x = y = g = 0;
            return true;
        }
        return false;
    }
    if (a == 0) {
        if (c % b == 0) {
            x = 0;
            y = c / b;
            g = abs(b);
            return true;
        }
        return false;
    }
    if (b == 0) {
        if (c % a == 0) {
            x = c / a;
            y = 0;
            g = abs(a);
            return true;
        }
        return false;
    }
    g = extgcd(a, b, x, y);
    if (c % g != 0) {
        return false;
    }
    T dx = c / a;
    c -= dx * a;
    T dy = c / b;
    c -= dy * b;
    x = dx + (T) ((__int128) x * (c / g) % b);
    y = dy + (T) ((__int128) y * (c / g) % a);
    g = abs(g);
    return true;
}
// |x|, |y| <= max(|a|, |b|, |c|) [tested]
}
```

```
bool crt(long long k1, long long m1, long long k2, long long m2
    , long long &k, long long &m) {
    k1 %= m1;
    if (k1 < 0) k1 += m1;
    k2 %= m2;
    if (k2 < 0) k2 += m2;
    long long x, y, g;
    if (!diophantine(m1, -m2, k2 - k1, x, y, g)) {
        return false;
    }
    long long dx = m2 / g;
    long long delta = x / dx - (x % dx < 0);
    k = m1 * (x - dx * delta) + k1;
    m = m1 / g * m2;
    assert(0 <= k && k < m);
    return true;
}

gauss.cpp
Description: Gaussian Elimination.
0b7335, 47 lines

const double eps = 1e-9;
int Gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = (int)a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
    double det = 1; int rank = 0;
    for(int col = 0, row = 0; col < m && row < n; ++col) {
        int mx = row;
        for (int i = row; i < n; i++) {
            if (fabs(a[i][col]) > fabs(a[mx][col])) { mx = i; }
        }
        if (fabs(a[mx][col]) < eps) { det = 0; continue; }
        for (int i = col; i <= m; i++) {
            swap(a[row][i], a[mx][i]);
        }
        if (row != mx) { det = -det; }
        det *= a[row][col];
        pos[col] = row;
        for (int i = 0; i < n; i++) {
            if (i != row && fabs(a[i][col]) > eps) {
                double c = a[i][col] / a[row][col];
                for (int j = col; j <= m; j++) {
                    a[i][j] -= a[row][j] * c;
                }
            }
        }
        ++row; ++rank;
    }
    ans.assign(m, 0);
    for(int i = 0; i < m; i++) {
        if (pos[i] != -1) { ans[i] = a[pos[i]][m] / a[pos[i]][i]
            ; }
    }
    for(int i = 0; i < n; i++) {
        double sum = 0;
        for (int j = 0; j < m; j++) {
            sum += ans[j] * a[i][j];
        }
        if(fabs(sum - a[i][m]) > eps) {
            return -1; //no solution
        }
    }
    for (int i = 0; i < m; i++) {
        if (pos[i] == -1) {
            return 2; //infinte solutions
        }
    }
    return 1; //unique solution
}
```

```
}

```

bin-gauss.cpp

Description: Gaussien Elimination modulo 2. n = number of equations, m = number of variables

39a607, 56 lines

```
int Gauss(int n, int m, vector<bitset<N>> a, bitset<N> &ans) {
    //reversing for lexicographically largest solution
    for (int i = 0; i < n; i++) {
        bitset<N> tmp;
        for (int j = 0; j < m; j++) tmp[j] = a[i][m - j - 1];
        tmp[m] = a[i][m];
        a[i] = tmp;
    }
    int rank = 0, det = 1;
    vector<int> pos(N, -1);
    for(int col = 0, row = 0; col < m && row < n; ++col) {
        int mx = row;
        for(int i = row; i < n; ++i) if(a[i][col]) { mx = i; break; }
        if(!a[mx][col]) { det = 0; continue; }
        swap(a[mx], a[row]);
        if (row != mx) { det = (det == 0 ? 0 : 1); }
        det &= a[row][col];
        pos[col] = row;
        //forward elimination
        for (int i = row + 1; i < n; ++i) {
            if (i != row && a[i][col]) {
                a[i] ^= a[row];
            }
        }
        ++row, ++rank;
    }
    ans.reset();
    //backward substitution
    for (int i = m - 1; i >= 0; i--) {
        if (pos[i] == -1) {
            ans[i] = true;
        } else {
            int k = pos[i];
            for (int j = i + 1; j < m; j++) if (a[k][j]) ans[i] = ans[i] ^ ans[j];
            ans[i] = ans[i] ^ a[k][m];
        }
    }
    for (int i = rank; i < n; ++i) {
        if (a[i][m]) {
            return -1; //no solution
        }
    }
    //reversing again beacuse we reversed earlier
    bitset<N> tmp;
    for (int j = 0; j < m; j++) {
        tmp[j] = ans[m - j - 1];
    }
    ans = tmp;
    int free_var = 0;
    for(int i = 0; i < m; ++i) {
        if (pos[i] == -1) {
            free_var++;
        }
    }
    return free_var; //has solution
}
```

mult-inv.cpp

Description: Calc multiplicative inverse.

93296c, 19 lines

```
int inv(int a, int m) {
```

bin-gauss mult-inv point polygon

```
    if (a == 1)
        return 1;
    return (1 - inv(m % a, a) * m) / a + m;
}

f[0] = 1;
for (int i = 1; i < N; i++) {
    f[i] = i * f[i - 1] % mod;
}
r[N - 1] = inv(f[N - 1]);
for (int i = N - 1; i >= 1; i--) {
    r[i - 1] = r[i] * i % mod;
}
//
r[1] = 1;
for (int i = 2; i < m; ++i) {
    r[i] = (m - (m / i) * r[m % i] % m) % m;
}

```

Geometry (7)

point.cpp

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

c2b847, 45 lines

```
template<typename T>
struct point_t {
    T x, y;
    point_t() : x(0), y(0) {}
    point_t(T _x, T _y) : x(_x), y(_y) {}
    T len_sq() const { return x * x + y * y; }
    ld len() const { return sqrtl(len_sq()); }
    point_t operator*(T k) const { return {x * k, y * k}; }
    void operator*=(T k) { x *= k; y *= k; }
    point_t operator+(const point_t<T> &other) const { return {
        x + other.x, y + other.y; } }
    point_t operator-(const point_t<T> &other) const { return {
        x - other.x, y - other.y; } }
    bool operator==(const point_t<T> &other) const { return x
        == other.x && y == other.y; }
    bool operator!=(const point_t<T> &other) const { return !(*
        this == other); }
    T operator*(const point_t<T> &other) const { return x *
        other.x + y * other.y; } // dot product
    T operator%(const point_t<T> &other) const { return x *
        other.y - y * other.x; } // cross product
};
using Point = point_t<ll>;

template<typename T>
istream &operator>>(istream &is, point_t<T> &vec) {
    is >> vec.x >> vec.y;
    return is;
}

template<typename T>
ostream &operator<<(ostream &os, const point_t<T> &vec) {
    os << vec.x << ' ' << vec.y;
    return os;
}

bool cmp_vectors(Point a, Point b) {
    if ((a.y < 0 || (a.y == 0 && a.x < 0)) && (b.y > 0 || (b.y
        == 0 && b.x > 0))) {
        return true;
    }
    if ((b.y < 0 || (b.y == 0 && b.x < 0)) && (a.y > 0 || (a.y
        == 0 && a.x > 0))) {

```

```
        return false;
    }
    return (a % b > 0 || (a % b == 0 && a.len_sq() < b.len_sq()
        ));
}

int get_sign(ld x) {
    if (x < -EPS) return -1;
    if (EPS < x) return 1;
    return 0;
}

```

polygon.cpp

Description: Polygon functions

09e4ae, 97 lines

```
using Polygon = vector<Point>;

Polygon build_convex_hull(Polygon &a) {
    int n = sz(a);
    for (int i = 1; i < n; ++i) {
        if ((a[i].y < a[0].y) || (a[i].y == a[0].y && a[i].x <
            a[0].x)) swap(a[0], a[i]);
    }
    sort(a.begin() + 1, a.end(), [&](Point A, Point B) {
        Point oa = A - a[0], ob = B - a[0];
        if ((oa % ob) == 0) return oa.len_sq() < ob.len_sq();
        return (oa % ob) > 0;
    });
    Polygon hull = {a[0]};
    for (int i = 1; i < n; ++i) {
        while (sz(hull) >= 2) {
            Point ab = hull[sz(hull) - 1] - hull[sz(hull) - 2],
                bp = a[i] - hull[sz(hull) - 1];
            if ((ab % bp) <= 0) {
                hull.pop_back();
            } else {
                break;
            }
        }
        hull.emplace_back(a[i]);
    }
    return hull;
}

ld area_of_polygon(Polygon &poly) {
    ll res = 0;
    int n = sz(poly);
    for (int i = 2; i < n; ++i) {
        Point ab = poly[i - 1] - poly[0], ac = poly[i] - poly
            [0];
        res += (ab % ac);
    }
    return abs(res) / 2;
}

ld perimeter_of_polygon(Polygon &poly) {
    ld res = 0;
    int n = sz(poly);
    for (int i = 0; i < n; ++i) {
        Point v = poly[(i + 1 == n ? 0 : i + 1)] - poly[i];
        res += v.len();
    }
    return res;
}

int point_in_polygon(const Point &p, const Polygon &poly) {
    // 0 - outside, 1 - inside, 2 - border;
    int n = sz(poly);
    if (point_in_angle(p, poly[n - 1], poly[0], poly[1])) {

```

```

int l = 1, r = n - 1;
while (r - l > 1) {
    int md = (l + r) / 2;
    if (point_in_angle(p, poly[md], poly[0], poly[l]))
    {
        r = md;
    } else {
        l = md;
    }
}
if (point_in_angle(p, poly[r], poly[0], poly[l]) &&
    point_in_angle(p, poly[0], poly[l], poly[r]) &&
    point_in_angle(p, poly[l], poly[r], poly[0])) {
    if (point_on_segment(p, poly[l], poly[r]) ||
        point_on_segment(p, poly[(l ? l - 1 : n - 1)],
            poly[l]) ||
        point_on_segment(p, poly[r], poly[(r + 1 == n ?
            0 : r + 1)])) {
        return 2;
    } else {
        return 1;
    }
} else {
    return 0;
}
}
}

int point_in_nonconvex_polygon(const Point &p, const Polygon &
    poly) {
    // 0 - outside, 1 - inside, 2 - border;
    int n = sz(poly);
    for (int i = 0; i < n; ++i) {
        if (point_on_segment(p, poly[i], poly[(i + 1) % n])) {
            return 2;
        }
    }
    ld s = 0.0;
    for (int i = 0; i < n; ++i) {
        Point pa = poly[i] - p, pb = poly[(i + 1 == n ? 0 : i +
            1)] - p;
        s += angle_rad(pa, pb);
    }
    if (s >= PI || s <= -PI) {
        return 1;
    } else {
        return 0;
    }
}
}

```

angle.cpp

Description: Some stuff with angles.

76aeac, 19 lines

```

template<typename T>
ld angle(const point_t<T> &a, const point_t<T> &b) {
    return fabs1(atan2(a % b, a * b) / PI * 180);
}

```

```

template<typename T>
ld angle_rad(const point_t<T> &a, const point_t<T> &b) {
    return (atan2(a % b, a * b));
}

```

```

template<typename T>
ld angle_rad(const point_t<T> &a) {
    return atan2(a.y, a.x);
}

```

```

template<typename T>
point_t<ld> rotate(const point_t<T> &a, ld alpha) {
    return {a.x * cos(alpha) - a.y * sin(alpha), a.x * sin(
        alpha) + a.y * cos(alpha)};
}

```

distance.cpp

Description: Calc the distance between everything.

8a21fc, 70 lines

```

ld from_point_to_line(const Point &p, const Point &a, const
    Point &b) { // point p, line ab
    Point ba = a - b, ap = p - a;
    return fabs((ba % ap) / ba.len());
}

ld from_point_to_ray(const Point &p, const Point &a, const
    Point &b) { // point p, ray ab
    Point ba = a - b, ap = p - a, ab = b - a;
    if ((ab * ap) < 0) {
        return ap.len();
    } else {
        return fabs((ba % ap) / ba.len());
    }
}

ld from_point_to_segment(const Point &p, const Point &a, const
    Point &b) { // point p, segment ab
    Point ab = b - a, ap = p - a, bp = p - b, ba = a - b;
    if ((ab * ap) < 0) {
        return fabs1(ap.len());
    } else if ((ab * bp) > 0) {
        return fabs1(bp.len());
    } else {
        return fabs1((ba % ap) / ba.len());
    }
}

ld from_segment_to_segment(const Point &a, const Point &b,
    const Point &c, const Point &d) { // segment ab, segment
    cd
    if (segment_intersection(a, b, c, d)) {
        return 0;
    }
    return min({from_point_to_segment(a, c, d),
        from_point_to_segment(b, c, d),
        from_point_to_segment(c, a, b),
        from_point_to_segment(d, a, b)});
}

ld from_segment_to_ray(const Point &a, const Point &b, const
    Point &c, const Point &d) { // segment ab, ray cd
    if (ray_segment_intersection(c, d, a, b)) {
        return 0;
    }
    return min({from_point_to_ray(a, c, d),
        from_point_to_ray(b, c, d),
        from_point_to_segment(c, a, b)});
}

ld from_segment_to_line(const Point &a, const Point &b, const
    Point &c, const Point &d) { // segment ab, line cd
    Point cd = d - c, ca = a - c, cb = b - c;
    if (get_sign(cd % ca) * get_sign(cd % cb) <= 0) {
        return 0;
    }
    return min(from_point_to_line(a, c, d), from_point_to_line(
        b, c, d));
}

```

```

ld from_ray_to_ray(const Point &a, const Point &b, const Point
    &c, const Point &d) { // ray ab, ray cd
    if (rays_intersection(a, b, c, d)) {
        return 0;
    }
    return min(from_point_to_ray(a, c, d), from_point_to_ray(c,
        a, b));
}

```

```

ld from_ray_to_line(const Point &a, const Point &b, const Point
    &c, const Point &d) { // ray ab, line cd
    if (line_ray_intersection(c, d, a, b)) {
        return 0;
    }
    return from_point_to_line(a, c, d);
}

```

```

ld from_line_to_line(const Point &a, const Point &b, const
    Point &c, const Point &d) { // line ab, line cd
    if (lines_intersection(a, b, c, d)) {
        return 0;
    }
    return from_point_to_line(a, c, d);
}

```

intersections.cpp

Description: Intersect everything.

59a62e, 101 lines

```

bool point_on_line(const Point &p, const Point &a, const Point
    &b) { // point p, line ab
    Point ab = b - a, ap = p - a;
    return (ab % ap) == 0;
}

bool point_on_ray(const Point &p, const Point &a, const Point &
    b) { // point p, ray ab
    Point ab = b - a, ap = p - a;
    return ((ab % ap) == 0 && (ab * ap) > 0) || (a == p);
}

bool point_on_segment(const Point &p, const Point &a, const
    Point &b) { // point p, segment ab
    Point ap = p - a, bp = p - b, ab = b - a;
    return (ap * bp) <= 0 && (ap % ab) == 0;
}

bool point_in_angle(const Point &p, const Point &a, const Point
    &o, const Point &b) { // point p, angle aob
    Point oa = a - o, ob = b - o, op = p - o;
    if ((oa % ob) < 0) swap(oa, ob);
    return ((oa % op) >= 0 && (ob % op) <= 0);
}

bool segment_intersection(const Point &a, const Point &b, const
    Point &c, const Point &d) { // segment ab, segment, cd
    Point ab = b - a, cd = d - c, ac = c - a, ad = d - a, cb =
        b - c, ca = a - c;
    if (get_sign((ab % ac)) * get_sign((ab % ad)) <= 0 &&
        get_sign((cd % ca)) * get_sign((cd % cb)) <= 0) {
        ll x1 = max(min(a.x, b.x), min(c.x, d.x)), x2 = min(max
            (a.x, b.x), max(c.x, d.x));
        ll y1 = max(min(a.y, b.y), min(c.y, d.y)), y2 = min(max
            (a.y, b.y), max(c.y, d.y));
        return (x1 <= x2 && y1 <= y2);
    }
    return false;
}

```

```
bool lines_intersection(const Point &a, const Point &b, const Point &c, const Point &d) { // line ab, line cd
    Point ab = b - a, cd = d - c, ac = c - a;
    return ((ab % cd) != 0 || (ab % ac == 0));
}

bool line_ray_intersection(const Point &a, const Point &b, const Point &c, const Point &d) { // line ab, ray cd
    if (!lines_intersection(a, b, c, d)) {
        return false;
    }
    Point ab = b - a, dp = d + (a - c), ac = c - a;
    if (get_sign(ab % (dp - a)) * get_sign(ab % ac) <= 0) {
        return true;
    }
    return false;
}

Point get_inf(const Point &a, const Point &b) { // returns inf point on ray ab
    if (a.x == b.x) {
        if (a.y < b.y) {
            return {a.x, INFL};
        } else {
            return {a.x, -INFL};
        }
    }
    if (a.x < b.x) {
        if (a.y < b.y) {
            return {INFL, INFL};
        } else if (a.y == b.y) {
            return {INFL, a.y};
        } else {
            return {INFL, -INFL};
        }
    }
    // a.x > b.x
    if (a.y < b.y) {
        return {-INFL, INFL};
    } else if (a.y == b.y) {
        return {-INFL, a.y};
    } else {
        return {-INFL, -INFL};
    }
}

bool rays_intersection(const Point &a, const Point &b, const Point &c, const Point &d) { // ray ab, ray cd
    if (line_ray_intersection(a, b, c, d) && line_ray_intersection(c, d, a, b)) {
        Point bp = get_inf(a, b), dp = get_inf(c, d);
        ll x1 = max(min(a.x, bp.x), min(c.x, dp.x)), x2 = min(max(a.x, bp.x), max(c.x, dp.x));
        ll y1 = max(min(a.y, bp.y), min(c.y, dp.y)), y2 = min(max(a.y, bp.y), max(c.y, dp.y));
        return (x1 <= x2 && y1 <= y2);
    }
    return false;
}

bool segment_line_intersection(const Point &a, const Point &b, const Point &c, const Point &d) { // segment ab, line cd
    Point cd = d - c, ca = a - c, cb = b - c;
    if (get_sign(cd % ca) * get_sign(cd % cb) <= 0) {
        return true;
    }
    return false;
}
```

```
bool ray_segment_intersection(const Point &a, const Point &b, const Point &c, const Point &d) { // ray ab, segment cd
    if (line_ray_intersection(c, d, a, b) && segment_line_intersection(c, d, a, b)) {
        Point bp = get_inf(a, b);
        ll x1 = max(min(a.x, bp.x), min(c.x, d.x)), x2 = min(max(a.x, bp.x), max(c.x, d.x));
        ll y1 = max(min(a.y, bp.y), min(c.y, d.y)), y2 = min(max(a.y, bp.y), max(c.y, d.y));
        return (x1 <= x2 && y1 <= y2);
    }
    return false;
}
```

diameter.cpp
Description: Rotating calipers.
Time: $\mathcal{O}(n)$

```
ll diameter_of_polygon(Polygon &poly) {
    int n = sz(poly), x = 1;
    ll ans = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        while (true) {
            Point ac = poly[x] - poly[i], ab = poly[j] - poly[i],
                  cd = poly[(x + 1) % n] - poly[x];
            ans = max(ans, ac.len_sq());
            if ((ab % cd) <= 0) {
                break;
            }
            x++;
            x %= n;
        }
        return sqrtl(ans);
    }
}
```

minkowski.cpp
Description: Minkowski sum of two polygons and its applications.
Time: $\mathcal{O}(|A| + |B|)$

```
Polygon minkowski_sum(Polygon &a, Polygon &b) {
    // a[0], b[0]: y - max, y1 = y2 => x - max. counter clockwise
    int n = sz(a), m = sz(b);
    assert(n >= 3 && m >= 3);
    Point high_a = a[0], high_b = b[0];
    Polygon va(n), vb(m);
    for (int i = 0; i < n; ++i) {
        va[i] = a[(i + 1) % n] - a[i];
    }
    for (int i = 0; i < m; ++i) {
        vb[i] = b[(i + 1) % m] - b[i];
    }
    // sort(all(va), cmp_vectors);
    // sort(all(vb), cmp_vectors);
    Polygon vc(sz(va) + sz(vb));
    merge(all(va), all(vb), vc.begin(), cmp_vectors);
    Point high_c(high_a.x + high_b.x, high_a.y + high_b.y);
    Polygon c(sz(vc) + 1);
    c[0] = high_c;
    for (int i = 0; i < sz(c) - 1; ++i) {
        c[i + 1] = c[i] + vc[i];
    }
    return c;
}

ld from_polygon_to_polygon(Polygon a, Polygon b) {
    for (auto &i : b) {
        i *= -1;
    }
}
```

```
}
int pos = 0;
for (int i = 1; i < sz(b); ++i) {
    if ((b[i].y > b[pos].y) || (b[i].y == b[pos].y && b[i].x > b[pos].x)) {
        pos = i;
    }
}
rotate(b.begin(), b.begin() + pos, b.end());
Polygon c = minkowski_sum(a, b);
int n = sz(c);
Point p(0, 0);
ld ans = 1e20;
for (int i = 0; i < n - 1; ++i) {
    ans = min(ans, from_point_to_segment(p, c[i], c[i + 1]));
}
return ans;
}

ld diameter_of_polygon_minkowski(Polygon &a) {
    Polygon ra = a;
    for (auto &i : ra) {
        i *= -1;
    }
    int pos = 0;
    for (int i = 1; i < sz(a); ++i) {
        if ((a[i].y > a[pos].y) || (a[i].y == a[pos].y && a[i].x > a[pos].x)) {
            pos = i;
        }
    }
    rotate(a.begin(), a.begin() + pos, a.end());
    pos = 0;
    for (int i = 1; i < sz(a); ++i) {
        if ((ra[i].y > ra[pos].y) || (ra[i].y == ra[pos].y && ra[i].x > ra[pos].x)) {
            pos = i;
        }
    }
    rotate(ra.begin(), ra.begin() + pos, ra.end());
    Polygon c = minkowski_sum(a, ra);
    int n = sz(c);
    ll ans = 0;
    for (int i = 0; i < n; ++i) {
        ans = max(ans, c[i].len_sq());
    }
    return sqrtl(ans);
}
```

tangents.cpp
Description: Returns id of tangent point: (left tangent, right tangent) (maybe (right, left), but i believe no).

```
pii tangent_from_point(const Point &p, const Polygon &poly) {
    int n = sz(poly);
    int i_min = 0;
    int i_max = 0;
    for (int k = 17; k >= 0; --k) {
        int l = (i_min + (1 << k)) % n;
        int r = ((i_min - (1 << k)) % n + n) % n;
        i_min = min({l, r, i_min}, [&poly, &p](int i, int j) {
            return ((poly[i] - p) % (poly[j] - p)) < 0 || ((poly[i] - p) % (poly[j] - p)) == 0 && (poly[i] - p).len_sq() < (poly[j] - p).len_sq();
        });
    }
}
```

```

{
    int l = (i_max + (1 << k)) % n;
    int r = ((i_max - (1 << k)) % n + n) % n;
    i_max = max({l, r, i_max}, [&poly, &p](int i, int j)
    ) {
        return ((poly[i] - p) % (poly[j] - p)) < 0 ||
            ((poly[i] - p) % (poly[j] - p)) == 0 &&
                (poly[i] - p).len_sq() > (poly[j] - p).len_sq());
    });
}

}
return {i_min, i_max};
}

```

circle.cpp

Description: Class to handle circle in the plane. T can be e.g. double or long long. (Avoid int.)

6e8a38, 55 lines

```

template<typename T>
struct circle_t {
    point_t<T> c;
    T r;

    circle_t() {}

    circle_t(point_t<T> _c, T _r) : c(_c), r(_r) {}

    int point_in(const point_t<T> &a) {
        if (is_same<T, ll>()) {
            ll dist = (a - c).len_sq();
            if (dist > r * r) {
                return 0;
            } else if (dist < r * r) {
                return 1;
            } else {
                return 2;
            }
        } else {
            ld dist = (a - c).len();
            if (dist > r + EPS) {
                return 0;
            } else if (dist + EPS < r) {
                return 1;
            } else {
                return 2;
            }
        }
    }

    vector<PointLD> tangent_from_point(const point_t<T> &a) {
        PointLD p(a.x, a.y);
        PointLD vec(c.x - p.x, c.y - p.y);
        ld dist = vec.len();
        if (dist + EPS < r) {
            return {};
        }
        if (abs(r - dist) < EPS) {
            return {p};
        }
        vec.x /= dist;
        vec.y /= dist;
        ld k = sqrtl(dist * dist - r * r);
        ld alpha = atan2(r, k);
        PointLD t1 = p + rotate(vec, alpha) * k, t2 = p +
            rotate(vec, -alpha) * k;
        return {t1, t2};
    }
};

```

```

template<typename T>
istream &operator>>(istream &is, circle_t<T> &c) {
    is >> c.c >> c.r;
    return is;
}

halfplanes.cpp
Description: half-plane intersection area
Time:  $\mathcal{O}(N \log N)$ 
9dca58, 104 lines

using Half = array<ld, 3>; // half-plane, ax + by >= c
using vH = vector<Half>;

PointLD hp_point(const Half &h) { return {h[0], h[1]}; } //
    direction of half-plane
PointLD isect(const Half &h0, const Half &h1) { // Cramer's
    rule to intersect half-planes
    array<ld, 3> vals{};
    for (int i = -1; i <= 1; ++i) {
        int x = (i == 0 ? 2 : 0), y = (i == 1 ? 2 : 1);
        vals[1 + i] = h0[x] * h1[y] - h0[y] * h1[x];
    }
    assert(fabssl(vals[0]) > EPS);
    return {vals[1] / vals[0], vals[2] / vals[0]};
}

ld eval(const Half &h, ld x) { // evaluate half-plane at x-
    coordinate
    assert(fabssl(h[1]) > EPS);
    return (h[2] - h[0] * x) / h[1];
}

ld x_isect(const Half &h0, const Half &h1) { return isect(h0,
    h1).x; } // x-coordinate of intersection

vH construct_lower(PointLD x, vH planes) { // similar to convex
    hull (by duality)
    sort(all(planes), [](const Half &a, const Half &b) {
        return hp_point(a) % hp_point(b) > EPS;
    });
    vH res{{1, 0, x.x}}; //>= x.f
    planes.push_back({-1, 0, -x.y}); //<= x.s
    auto lst_x = [&](Half a, Half b) {
        if (fabssl(hp_point(a) % hp_point(b)) <= EPS) { //
            parallel half-planes, remove lower one
            return a[2] / a[1] <= b[2] / b[1] ? x.x : x.y;
        }
        return x_isect(a, b);
    };
    for (auto t : planes) {
        while (sz(res) > 1 && lst_x(res.back(), t) <= lst_x(res
            [sz(res) - 2], res.back())) {
            res.pop_back();
        }
        res.push_back(t);
    }
    return res;
}

ld isect_area(vH planes) {
    const ld BIG = 1e9;
    PointLD x{-BIG, BIG};
    planes.push_back({0, 1, -BIG}); // y >= -BIG
    planes.push_back({0, -1, -BIG}); // -y >= -BIG
    vH upper, lower;
    for (auto &t : planes) {
        if (fabssl(t[1]) <= EPS) { // vertical line
            ld quo = t[2] / t[0];

```

```

            if (t[0] > 0) {
                if (quo > x.x) x.x = quo;
            } else { // -x >=
                if (quo < x.y) x.y = quo;
            }
        } else if (t[1] > 0) {
            lower.push_back(t);
        } else {
            upper.push_back(t);
        }
    }
    if (x.x >= x.y) return 0;
    lower = construct_lower(x, lower);
    for (auto &t : upper) {
        t[0] *= -1;
        t[1] *= -1;
    }
    upper = construct_lower({-x.y, -x.x}, upper);
    for (auto &t : upper) {
        t[0] *= -1;
        t[1] *= -1;
    }
    reverse(all(upper));
    int iu = 1, il = 1;
    ld lst = x.x, lst_dif = eval(upper[1], lst) - eval(lower
        [1], lst);
    ld ans = 0;
    while (iu < sz(upper) - 1 && il < sz(lower) - 1) { // sweep
        vertical line through lower and upper hulls
        ld nex_upper = x_isect(upper[iu], upper[iu + 1]);
        ld nex_lower = x_isect(lower[il], lower[il + 1]);
        ld nex = min(nex_upper, nex_lower);
        ld nex_dif = eval(upper[iu], nex) - eval(lower[il], nex
            );
        auto avg_val = [](ld a, ld b) -> ld {
            if (a > b) swap(a, b);
            if (b <= 0) return 0;
            if (a >= 0) return (a + b) / 2;
            return b / (b - a) * b / 2;
        };
        ans += (nex - lst) * avg_val(lst_dif, nex_dif);
        assert(x.x <= nex && nex <= x.y);
        lst = nex, lst_dif = nex_dif;
        iu += fabssl(lst - nex_upper) <= EPS;
        il += fabssl(lst - nex_lower) <= EPS;
    }
    return ans;
}

Half plane_right(PointLD a, PointLD b) { // half-plane to right
    of a -> b
    return {b.y - a.y, a.x - b.x, (b.y - a.y) * a.x + (a.x - b.
        x) * a.y};
}

Half plane_through(PointLD p, PointLD dir) { // half-plane
    through p in direction dir
    return {dir.x, dir.y, p * dir};
}

```

Задача	Статус	Вася	Дима	Саша	Идеи
А - 1					
В - 2					
С - 3					
Д - 4					
Е - 5					
F - 6					
G - 7					
Н - 8					
I - 9					
J - 10					
К - 11					
L - 12					
М - 13					
N - 14					
О - 15					